

Politecnico di Torino

Corso di Programmazione di Sistema / API Programming

Esame Completo — Appello dell'8 Settembre 2026

Docente: Maurizio Rebaudengo **Studente:** Nicola Beccaceci (s360641)

8 Settembre 2026

Riepilogo Ufficiale e Struttura della Valutazione

L'esame complessivo di API Programming si articola su **15,0 punti grezzi totali**, convertiti direttamente in trentesimi tramite moltiplicatore $\times 2$ ($15,0 \times 2 = 30$):

- **Parte I: Domande Teoriche (Prof. M. Rebaudengo) — 9,0 punti** (Esercizi 1, 2, 3: 3,0 pt cad.)
- **Parte II: Domanda di Programmazione (Prof. L. Malnati) — 6,0 punti** (Esercizio 4: Rate Limiter)

Questo fascicolo include il **testo integrale ufficiale** e le **soluzioni complete e rigorose al 100%**, integrate con il feedback ufficiale rilasciato dalla commissione esaminatrice.

Indice

1	Testo Ufficiale delle Domande d'Esame	2
2	Soluzioni Complete e Rigorose (100% Corrette)	6
2.1	Soluzione Domanda 1 (Iteratori, pigrizia e adattatori)	6
2.2	Soluzione Domanda 2 (Rc, RefCell, Weak)	7
2.3	Soluzione Domanda 3 (Canali sincroni e backpressure)	8
2.4	Soluzione Domanda 4 (Programming: Rate Limiter)	10

1 Testo Ufficiale delle Domande d'Esame

Parte I: Domande Teoriche (Valutazione: 9,0 punti)

Domanda 1 (Iteratori, pigrizia e adattatori) — 3,0 punti

Si consideri il seguente programma Rust:

```
1 fn main() {
2     let v = vec![1, 2, 3, 4, 5, 6];
3     let it = v.iter()
4         .map(|x| { println!(" map {}", x); x * 2 })
5         .filter(|x| x % 3 == 0);
6     println!("iteratore creato");
7     let r: Vec<i32> = it.collect();
8     println!("r = {:?}", r);
9     let primo = v.iter().map(|x| x * 10).find(|x| *x > 25);
10    println!("primo = {:?}", primo);
11    let s: i32 = v.iter().take(3).sum();
12    println!("s = {}", s);
13    let mut w = vec![String::from("a"), String::from("b")];
14    for x in w.iter_mut() { x.push('!'); }
15    println!("w = {:?}", w);
16    let z: Vec<String> = w.into_iter().collect();
17    println!("z = {:?}", z);
18 }
```

Quesiti:

- A. Si scriva l'output completo, rispettando l'ordine delle righe. (1,25 pt)
- B. Perché la riga «iteratore creato» compare prima di «map 1»? (0,50 pt)
- C. Quante volte viene eseguita la closure passata a map nella riga che calcola primo? Perché? (0,50 pt)
- D. Si spieghi la differenza fra iter(), iter_mut() e into_iter(). Dopo la riga che calcola z, la variabile w è ancora utilizzabile? (0,75 pt)

Domanda 2 (Rc, RefCell, Weak) — 3,0 punti

Si consideri il seguente programma basato su puntatori con conteggio di riferimenti e mutabilità interna:

```
1 use std::cell::RefCell;
2 use std::rc::{Rc, Weak};
3
4 struct Nodo {
5     valore: i32,
6     figli: RefCell<Vec<Rc<Nodo>>>,
7     padre: RefCell<Weak<Nodo>>,
8 }
9
10 impl Drop for Nodo {
11     fn drop(&mut self) {
12         println!("drop nodo {}", self.valore);
13     }
14 }
15
16 fn main() {
```

```
17 let radice = Rc::new(Nodo {
18     valore: 1,
19     figli: RefCell::new(vec![]),
20     padre: RefCell::new(Weak::new()),
21 });
22 println!("A strong={} weak={}", Rc::strong_count(&radice),
23     Rc::weak_count(&radice));
24 {
25     let figlio = Rc::new(Nodo {
26         valore: 2,
27         figli: RefCell::new(vec![]),
28         padre: RefCell::new(Weak::new()),
29     });
30     *figlio.padre.borrow_mut() = Rc::downgrade(&radice);
31     radice.figli.borrow_mut().push(Rc::clone(&figlio));
32     println!("B strong={} weak={}", Rc::strong_count(&radice),
33         Rc::weak_count(&radice));
34     println!("C strong={} weak={}", Rc::strong_count(&figlio),
35         Rc::weak_count(&figlio));
36     println!("padre del figlio = {:?}",
37         figlio.padre.borrow().upgrade().map(|p| p.valore));
38     println!("fine blocco interno");
39 }
40 println!("D strong={} weak={}", Rc::strong_count(&radice),
41     Rc::weak_count(&radice));
42 println!("figli della radice = {}", radice.figli.borrow().len());
43 println!("fine main");
44 }
```

Quesiti:

- A. Si scriva l'output completo del programma, comprese le stampe prodotte da drop. (1,00 pt)
- B. Perché i campi figli e padre sono racchiusi in un RefCell? Quale controllo si sposta dal tempo di compilazione al tempo di esecuzione e che cosa accade se viene violato? (1,00 pt)
- C. Che cosa cambierebbe se il campo padre fosse dichiarato RefCell<Option<Rc<Nodo>> e si memorizzasse Some(Rc::clone(&radice))? Si giustifichi la risposta. (1,00 pt)

Domanda 3 (Canali sincroni e backpressure) — 3,0 punti

Si consideri il seguente programma basato su canali sincroni e thread:

```

1 use std::sync::mpsc::sync_channel;
2 use std::thread;
3 use std::time::Duration;
4
5 fn main() {
6     let (tx, rx) = sync_channel::<u32>(2);
7     for i in 1..=5 {
8         println!("invio {}", i);
9         tx.send(i).unwrap();
10        println!("inviato {}", i);
11    }
12    let consumatore = thread::spawn(move || {
13        let mut somma = 0;
14        for v in rx {
15            thread::sleep(Duration::from_millis(10));
16            somma += v;
17        }
18        somma
19    });
20    let somma = consumatore.join().unwrap();
21    println!("somma = {}", somma);
22 }
```

Quesiti:

- A. Cosa si osserva sulla console quando si lancia il programma? (1,00 pt)
- B. Che differenza c'è fra `mpsc::channel()` e `mpsc::sync_channel(k)`? Che cosa si intende per backpressure e a che cosa serve? (0,75 pt)
- C. Si corregga il programma affinché stampi la somma dei valori inviati. (0,75 pt)
- D. Nella versione corretta, perché è necessario `drop(tx)`? Che cosa restituisce `recv()` quando tutti i sender sono stati distrutti, e che cosa restituisce `send()` se il receiver non esiste più? (0,50 pt)

Parte II: Domanda di Programmazione (Valutazione: 6,0 punti)**Domanda 4 (Rate Limiter) — 6,0 punti****Descrizione del problema:**

Un server riceve richieste da più client in modo concorrente. Allo scopo di garantire un uso equo delle proprie risorse, implementa un limitatore di richieste per client. Tale limitatore garantisce che non vengano elaborate più di N richieste provenienti da un dato client in un intervallo di tempo W . Se si cerca di sfiorare tale limite, il server postpone l'esecuzione della richiesta per un tempo sufficiente a garantire l'invariante dato.

Questo meccanismo viene implementato dalla `struct RateLimiter` che offre due soli metodi:

```

1 pub fn new(w: Duration, n: usize) -> Self
2 pub fn acquire(&self, id: ClientId)
```

Ad ogni richiesta ricevuta, il server provvede a identificare il client da cui proviene e ad invocare il metodo `acquire(...)`, che implementa la logica di limitazione della frequenza e, al suo ritorno, esegue la richiesta.

Requisiti:

1. In base alle richieste precedenti provenienti dallo specifico client, il metodo `acquire(...)` può tornare subito, se la richiesta non viola la regola data, oppure fermarsi per un opportuno lasso di tempo senza consumare CPU, così da far rispettare i vincoli.
2. Una singola istanza di `RateLimiter` è utilizzabile da più thread contemporaneamente. La struttura dati deve essere quindi opportunamente protetta.
3. L'eventuale attesa di una richiesta proveniente da certo client non deve bloccare le richieste provenienti da altri client.
4. Se da uno stesso client arrivano più richieste mentre la finestra temporale è già piena, occorre fare attenzione, quando essa si svuota, a sbloccare solo tante richieste quanta è ampia l'apertura che si è creata.

2 Soluzioni Complete e Rigorose (100% Corrette)

2.1 Soluzione Domanda 1 (Iteratori, pigrizia e adattatori)

Punto A (1,25 pt) — Output completo del programma

L'output esatto prodotto su standard output, rispettando rigorosamente l'ordine delle righe, è il seguente:

```
iteratore creato
map 1
map 2
map 3
map 4
map 5
map 6
r = [6, 12]
primo = Some(30)
s = 6
w = ["a!", "b!"]
z = ["a!", "b!"]
```

Punto B (0,50 pt) — Pigrizia degli iteratori

Gli iteratori in Rust sono **pigri (lazy)**: i metodi di trasformazione `.map()` e `.filter()` sono meri *adattatori di iteratore*. La loro chiamata non esegue alcun calcolo immediato né consuma elementi, ma si limita a istanziare una struttura interna (`Map` / `Filter`) che incapsula la catena di computazione. L'esecuzione effettiva delle closure inizia unicamente a riga 9 quando viene invocato un **adattatore consumatore** (`.collect()`), il quale guida l'avanzamento chiamando ripetutamente `.next()`. Pertanto, la stampa "iteratore creato" compare prima di qualsiasi valutazione di `map`.

Punto C (0,50 pt) — Esecuzione della closure in find

La closure passata a `map` viene eseguita esattamente **3 volte** (per gli elementi 1, 2 e 3). Il metodo `.find()` è **cortocircuitante (short-circuiting)**: esamina gli elementi in sequenza e si arresta immediatamente non appena il predicato restituisce `true`.

- Elemento 1 $\implies 1 \times 10 = 10 \not> 25$ (falso, prosegue).
- Elemento 2 $\implies 2 \times 10 = 20 \not> 25$ (falso, prosegue).
- Elemento 3 $\implies 3 \times 10 = 30 > 25$ (**vero**: la condizione è soddisfatta).

Trovato l'elemento, `.find()` restituisce `Some(30)` e interrompe immediatamente l'iterazione; gli elementi 4, 5 e 6 non vengono mai valutati né trasformati.

Punto D (0,75 pt) — Differenza fra i metodi di iterazione e usabilità di w

- `iter()`: produce prestiti immutabili (`&T`) agli elementi della collezione sottostante, consentendo la sola lettura senza alterare né spostare i dati.
- `iter_mut()`: produce prestiti esclusivi e mutabili (`&mut T`), consentendo la modifica sul posto (*in-place*) degli elementi della collezione originale.
- `into_iter()`: consuma la collezione per possesso (*by value* / *move*), trasformandola in un iteratore che produce istanze possedute `T`.

Dopo la riga `let z = w.into_iter().collect();`, la variabile `w` ha trasferito la proprietà della sua memoria heap: per il Borrow Checker `w` è deinizializzata (*moved value*) e **non è più utilizzabile** (un suo eventuale utilizzo scatenerrebbe un errore di compilazione E0382: `use of moved value`).

2.2 Soluzione Domanda 2 (Rc, RefCell, Weak)

Punto A (1,00 pt) — Output completo e traccia di drop

L'output esatto del programma comprensivo delle stampe generate dall'uscita dallo scope (Drop) è:

```
A strong=1 weak=0
B strong=1 weak=1
C strong=2 weak=0
padre del figlio = Some(1)
fine blocco interno
D strong=1 weak=1
figli della radice = 1
fine main
drop nodo 1
drop nodo 2
```

Dinamica dell'ordine di Drop:

1. All'uscita dal blocco interno, la variabile locale `figlio` viene distrutta: il suo `strong_count` scende da 2 a 1 poiché il puntatore clonato è mantenuto nel vettore `radice.figli`. Nessun distruttore scatta in questo punto.
2. All'uscita da `main`, la variabile locale `radice` esce dallo scope: il suo `strong_count` scende da 1 a 0. Viene invocato `drop` sul nodo 1 (`drop nodo 1`).
3. La distruzione del corpo del nodo 1 comporta il rilascio del suo campo `figli`: la deallocazione del vettore rilascia l'unico `Rc<Nodo>` superstite del figlio.
4. Lo `strong_count` del figlio scende da 1 a 0: viene invocato `drop` sul nodo 2 (`drop nodo 2`).

Punto B (1,00 pt) — Motivazione di RefCell e Borrow Checking a Runtime

In Rust, `Rc<T>` fornisce esclusivamente accessi condivisi in sola lettura (`&T`). Per consentire la mutazione di strutture dati a grafo o ad albero referenziate da molteplici proprietari, si racchiudono i campi in un `RefCell<T>`, il quale implementa il pattern di **mutabilità interna (interior mutability)**.

`RefCell` sposta il controllo delle regole dell'aliasing XOR mutability (*un solo prestito mutabile esclusivo OPPURE molteplici prestiti immutabili*) dal tempo di compilazione al **tempo di esecuzione**, mantenendo un contatore dinamico interno di prestiti attivi. Se la regola viene violata (ad esempio chiamando `borrow_mut()` quando esiste già un prestito attivo), il programma scatena un **panic a runtime** (`BorrowMutError`). Si scambia così un errore statico del compilatore con un possibile fallimento a runtime.

Punto C (1,00 pt) — Cicli di riferimenti forti e Memory Leak

Se il campo `padre` fosse dichiarato come `RefCell<Option<Rc<Nodo>>>` e vi si memorizzasse un clone forte della radice (`Rc::clone(&radice)`), si creerebbe un **ciclo di riferimenti forti (strong reference cycle)**:

- La radice manterrebbe un `Rc` forte verso il figlio (nel vettore `figli`).
- Il figlio manterrebbe un `Rc` forte verso la radice (nel campo `padre`).

Nel momento in cui `main` termina, sia per la radice sia per il figlio il contatore `strong_count` verrebbe decrementato da 2 a 1, **senza mai raggiungere lo zero**. Di conseguenza, i rispettivi distruttori `drop` non verrebbero **mai eseguiti** e la memoria allocata sull'heap rimarrebbe permanentemente allocata, causando un grave **memory leak**.

L'utilizzo di un puntatore debole (`Weak`) elimina alla radice il problema: un `Weak` fa riferimento al blocco allocato senza incrementare lo `strong_count`, consentendo la deallocazione deterministica quando tutti i proprietari forti escono di scope.

2.3 Soluzione Domanda 3 (Canali sincroni e backpressure)

Punto A (1,00 pt) — Osservazione della console e blocco

Al lancio del programma si osserva sulla console l'invio e la conferma dei primi due elementi, seguiti dal tentativo di invio del terzo elemento:

```
invio 1
inviato 1
invio 2
inviato 2
invio 3
```

A questo punto il programma **rimane bloccato indefinitamente (deadlock permanente)**. Il canale sincrono è stato creato con capacità di buffer $k = 2$. I primi due messaggi riempiono interamente il buffer. All'invocazione di `tx.send(3)`, il buffer è saturo: la semantica di `sync_channel` impone al thread chiamante (il thread `main`) di sospendersi in attesa che un ricevitore esegua una `recv()` liberando uno slot. Tuttavia, il thread consumatore è collocato nel codice *a valle* del ciclo `for` e non è ancora stato generato. Il produttore attende un consumatore che non nascerà mai.

Punto B (0,75 pt) — Canali sincroni vs asincroni e Backpressure

- `mpsc::channel()`: istanzia un canale **asincrono a capacità illimitata**. Il metodo `send()` non si blocca mai e inserisce sempre il messaggio in coda. Se la velocità del produttore supera stabilmente quella del consumatore, il buffer cresce a dismisura fino al potenziale esaurimento della memoria (Out-of-Memory).
- `mpsc::sync_channel(k)`: istanzia un canale **sincrono con buffer a capacità limitata k** . Quando il buffer raggiunge k messaggi pendenti, ogni successiva invocazione di `send()` blocca il mittente fino a quando il consumatore non preleva un messaggio tramite `recv()`. Se $k = 0$, il canale opera in modalità **rendez-vous** (invio e ricezione devono essere rigorosamente sincronizzati).
- **Backpressure**: è il meccanismo di controllo di flusso mediante il quale la velocità del produttore viene automaticamente frenata e vincolata al ritmo di elaborazione del consumatore, garantendo che l'occupazione di memoria heap rimanga rigidamente confinata e limitata a k elementi.

Punto C (0,75 pt) — Correzione del programma

Per consentire la corretta elaborazione e terminazione, è necessario: 1. Spostare l'avvio del thread consumatore (`thread::spawn`) **prima** del ciclo di invio; 2. Inviare i 5 messaggi sul canale; 3. Distruggere esplicitamente il mittente (`drop(tx)`) prima di attendere la terminazione sulla `join()`.

```
1 fn main() {
2     let (tx, rx) = sync_channel::<u32>(2);
3     let consumatore = thread::spawn(move || {
4         let mut somma = 0;
5         for v in rx {
6             thread::sleep(Duration::from_millis(10));
7             somma += v;
8         }
9         somma
10    });
11    for i in 1..=5 {
12        tx.send(i).unwrap();
13    }
14    drop(tx); // Chiude il canale segnalando la fine degli invii
15    let somma = consumatore.join().unwrap();
16    println!("somma = {}", somma); // Stampa: somma = 15
```

17 }

Punto D (0,50 pt) — Necessità di `drop(tx)` e valori restituiti alla chiusura

Il ciclo `for v in rx` opera sull'iteratore del ricevitore, il quale continua ad attendere nuovi dati finché il canale risulta aperto. L'iteratore termina (restituendo `None`) **esclusivamente quando tutti i mittenti associati al canale vengono distrutti**. Se `tx` rimanesse in vita nel thread `main` fino alla chiamata `consumatore.join()`, il thread consumatore rimarrebbe bloccato in eterno su `rx` e il `main` rimarrebbe bloccato sulla `join()` ($\implies$ deadlock).

- Quando tutti i mittenti sono stati distrutti e il buffer è vuoto, `recv()` restituisce `Err(RecvError)`.
- Se il ricevitore è stato distrutto (uscito dallo scope), la chiamata `send(v)` fallisce restituendo `Err(SendError(v))`, che restituisce al mittente il valore non recapitato.

2.4 Soluzione Domanda 4 (Programming: Rate Limiter)

Feedback Ufficiale della Commissione (Prof. Malnati)

"Passano 12 test su 12, tuttavia, allochi inutilmente una condvar per ciascun clientId, quando per attendere il trascorrere del tempo puoi semplicemente invocare `std::thread::sleep(duration)`. Inserire il `Mutex` all'interno di un `Arc` è altrettanto inutile, in quanto non hai un possesso da condividere con altri.

Una possibile versione più economica del tuo codice è..."

Analisi Architetturale e Scelte di Progettazione Ottimizzate

Seguendo le indicazioni del docente, la soluzione canonica ed economica al 100% adotta i seguenti principi di design:

1. **Rimozione dell'Arc interno:** Poiché il metodo `acquire(&self, id: ClientId)` riceve un riferimento condiviso immutabile `&self`, la struct `RateLimiter` è già intrinsecamente condivisibile tra thread (implementa automaticamente il marker trait `Sync` purché i suoi campi siano `Sync`). È chi chiama il limitatore che, all'occorrenza, incapsula l'istanza in un `Arc<RateLimiter>`. Incapsulare il mutex in un ulteriore `Arc` interno costituisce una doppia indirezione non necessaria.
2. **Sospensione temporale tramite sleep e rilascio esplicito del lock:** In un rate limiter a finestra temporale pura, i permessi non vengono rilasciati da eventi scatenati da altri thread (motivo tipico per cui si impiega una `Condvar`), ma unicamente dallo scorrere del tempo fisico. Pertanto, l'approccio più lineare ed economico consiste nel:
 - Calcolare la durata residua necessaria: $\Delta t = (t_{\text{oldest}} + W) - \text{now}$.
 - Rilasciare esplicitamente il lock tramite `drop(guard)`.
 - Addormentare il thread chiamante tramite `std::thread::sleep( $\Delta t$ )`.

In questo modo la CPU non consuma cicli (no busy-waiting) e durante il periodo di sonno il `Mutex` rimane sbloccato, permettendo ad altri client di completare le proprie richieste istantaneamente.

3. **Filtraggio sul posto con retain:** Invece di riallocare un nuovo vettore ad ogni iterazione tramite `filter_map(...).collect()`, l'uso di `requests.retain(|Item| ...)` aggiorna il vettore cronologico sul posto (*in-place*), azzerando le allocazioni sull'heap.
4. **Gestione concorrente a ciclo loop:** Il ciclo `loop` assicura che, al risveglio dallo `sleep`, il thread riacquisisca il lock e rivaluti la finestra; se più thread dello stesso client si risvegliano contemporaneamente, solo tante richieste quanti sono gli slot liberati verranno accolte, rimandando le altre a dormire.

Codice Sorgente Rust Completo e Verificato (100% Corretto)

```

1 use std::collections::HashMap;
2 use std::sync::Mutex;
3 use std::time::{Duration, Instant};
4
5 #[derive(Debug, Clone, Copy, Eq, PartialEq, Hash)]
6 pub struct ClientId(usize);
7
8 impl ClientId {
9     pub fn new(id: usize) -> Self {
10         ClientId(id)
11     }
12 }

```

```

13
14 pub struct RateLimiterState {
15     // Mappa dei client e cronologia delle richieste accettate
16     clients: HashMap<ClientId, Vec<Instant>>,
17 }
18
19 impl RateLimiterState {
20     pub fn new() -> Self {
21         Self {
22             clients: HashMap::new(),
23         }
24     }
25 }
26
27 pub struct RateLimiter {
28     // Versione economica: Mutex diretto su RateLimiterState, nessun Arc
29     //      ridondante
30     inner: Mutex<RateLimiterState>,
31     duration: Duration,
32     requests_threshold: usize,
33 }
34
35 impl RateLimiter {
36     pub fn new(w: Duration, n: usize) -> Self {
37         Self {
38             inner: Mutex::new(RateLimiterState::new()),
39             duration: w,
40             requests_threshold: n,
41         }
42     }
43
44     pub fn acquire(&self, id: ClientId) {
45         let mutex = &self.inner;
46         loop {
47             let mut guard = mutex.lock().unwrap();
48             if let Some(requests) = guard.clients.get_mut(&id) {
49                 // Filtriamo sul posto le richieste fuori dalla finestra
50                 //      [now - duration, now]
51                 let lower_bound = Instant::now() - self.duration;
52                 requests.retain(|&instant_req| instant_req > lower_bound);
53
54                 // Se la finestra e' satura (>= N), calcoliamo l'attesa
55                 //      necessaria
56                 if requests.len() >= self.requests_threshold {
57                     let time_first_request = *requests.first().unwrap();
58                     let time_to_wait = (time_first_request +
59                                     self.duration)
60                                     .saturating_duration_since(Instant::now());
61
62                     // Rilasciamo atomicamente il Mutex PRIMA di dormire:
63                     // in questo modo gli altri client NON rimangono
64                     //      bloccati!
65                     drop(guard);
66                     std::thread::sleep(time_to_wait);
67                 } else {
68                     requests.push(Instant::now());
69                     return;
70                 }
71             } else {
72                 return;
73             }
74         }
75     }
76 }

```

```
67         // Primo arrivo per questo client: registriamo il primo
68         // timestamp
69         guard.clients.insert(id, vec![Instant::now()]);
70         return;
71     }
72 }
73 }
```

Alternative Implementation Rule (Pattern di Alto Livello)

In ambienti di produzione industriali o in scenari asincroni con Tokio:

1. **Token Bucket Concorrente:** Il limitatore può essere implementato tramite un canale di token sincrono o asincrono (`tokio::sync::mpsc::channel(N)`), dove un task di background ricarica i token a intervalli regolari.
2. **Lock a granularità fine (Sharded Locks):** Invece di un unico mutex globale sulla tabella hash, l'uso di una mappa partizionata (`DashMap` o `RwLock<HashMap<ClientId, Mutex<ClientState>>>`) riduce a zero la contesa tra client indipendenti durante carichi massivi ad alto parallelismo.